* ArrayList in Java

- Arraylist is a inbuilt property in Java. and it is a linear structure. (it's a collection framework)

import java.util.ArrayList; before use.

Syntax:

```
ArrayList<Integer> list =
  new ArrayList<>();
```

Array	ArrayList
fixed size	dynamic size
primitive data types can be stored.	primitive data types can not be stored.

→ Size of AL

list.size();

→ To pick elements

list.get(); //get function.

→ To set any value by picking from an array.

list.set(idx1, list.get(idx2));

To insert elements into arraylist

list.add(1); list.add(3);
list.add(2); list.add(4);

→ ArrayList operations

use Collections.sort()

- add
- Get
- modify
- delete / remove
- iterate / operations.

List = [0, 2, 3]; List.add(1, 1);

index — element you want to add

list = [0, 1, 2, 3];

To add element in between.

int	Integer
• primitive data type	wrapper class (object) for int
• It stores the actual numeric value.	stores reference to an object containing the value.
• int cannot be NULL	Integer can be NULL.
• int default value '0'.	default value is null
• faster memory efficient stored in a stack	slower & bigger (object in heap & reference in stack)

→ Sorting an ArrayList

- collections.sort(); // it will sort ascending order
- collections.sort(list, Collections.reverseOrder()); // descending

→ ArrayList implementation :

when you write ArrayList<Integer> list = new ArrayList<>();

in memory | 1 | 2 | 3 | 4 | 5 | when created array has filled

Java ditects & creates one more array with x2 size of previous.

| 1 | 2 | 3 | 4 | 5 | | | | | | & copies all the elements like wise

it works. privious array will be discarded

→ multidimensional ArrayList :

```
ArrayList<ArrayList<Integer>> mainList = new ArrayList<>();
// two arrayList because they act as a row & column
ArrayList<Integer> list1 = new ArrayList<>();
list1.add(1); list1.add(2);
mainList.add(list1);
```

Same thing repeat for list2

```
// to print
for(int i=0; i< mainList.size(); i++)
{
    ArrayList<Integer> curList = mainList.get(i);
    for(int j=0; j<curList.size(); j++)
    {
        System.out.print(curList.get(j) + " ");
    }
    System.out.println();
}
```

Math.min( ); // to find min value.

Math.max( )

* Container with more water (using arraylist)

```
public static int storeWater(ArrayList<Integer> height){
    int maxWater = 0;
    int lp = 0;
    int rp = height.size() - 1;
    while( lp < rp){
        int ht = Math.min(height.get(lp), height.get(rp));
        int width = RP - LP;
        int curWater = ht * width;
        maxWater = Math.max(maxWater, curWater);
        if (lp < rp)
        {
            lp++;
        }
        else{
            rp--;
        }
    }
    return maxWater;
}
```

1	8	6	2	5	4	8	3	7

= maxwater = 49

O(n) linear

2 pointer approach

* pair sum using (Two pointer)

```
boolean pairSum(ArrayList<Integer> height){
                int target
    int lp = 0;
    int rp = height.size() - 1;
    while(lp < rp){
        int temp = height.get(lp) + height.get(rp);
        if (temp == target)
        {
            return true;
        }
        if( temp > target)
        {
            rp--;
        }
        else{
            lp++;
        }
    }
    return false;
}
```

Here array List is Sorted

1	2	3	4	5	6

target = 8

1, 4 2, 3

O(n).

* Array and ArrayList

Array is a collection of similar items / same datatype. cannot possible to hold different type values in a single array in Java. array is a contiguous memory location.

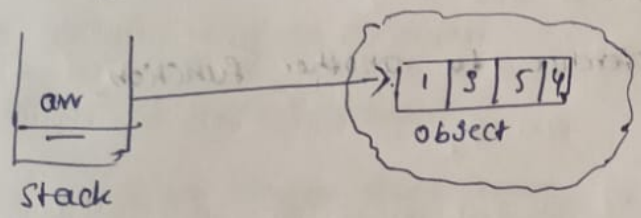
int[] A ; // it is just a declaration means getting defined in the stack.

A = new int[5]; // initialisation creating the object in heap memory.

-> declaration done during compile time.

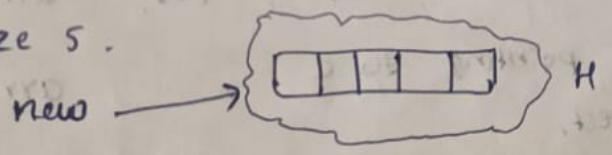
-> initialisation done during runtime. (dynamic memory allocation, during runtime allocating memory).

-> heap objects (inside heap objects) are not contiguous, no proof. we know array objects are in heap so in Java array may not be continuous. because that depends on JVM.



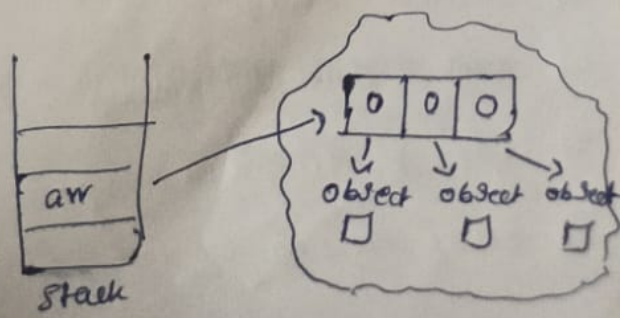
-> basically new keyword used to create an object.

ex: new int[5]; // new will create the array in heap (object) of type integer & size 5.



-> in python everything is object. But in Java primitive / reference store in stack. and objects store in heap memory.

-> all the objects store in a different memory location inside heap.



```
int[] A = new int[5];
```

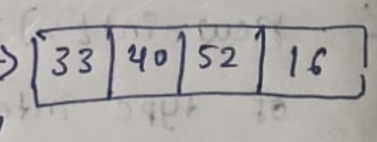
43	20	31	29	80
0	1	2	3	4

```
for(int i=0; i<A.length; i++){ // normal process to print arr elements.
    S.o.p(A[i] +" ");
}

for(int a : A){ //enhanced for each loop
    S.o.p(a+" ");
}

S.o.p(Arrays.toString(A)); // passing entire array, it will convert to String & prints.
```

→ we can also change arr values pass by reference by another new function.

```
P.S.V.main(){
    int[] A = {33, 40, 52, 16};
    change(A); // passing the reference to another function.
}
static void change(int[] arr){
    arr[1] = 90;
}
```

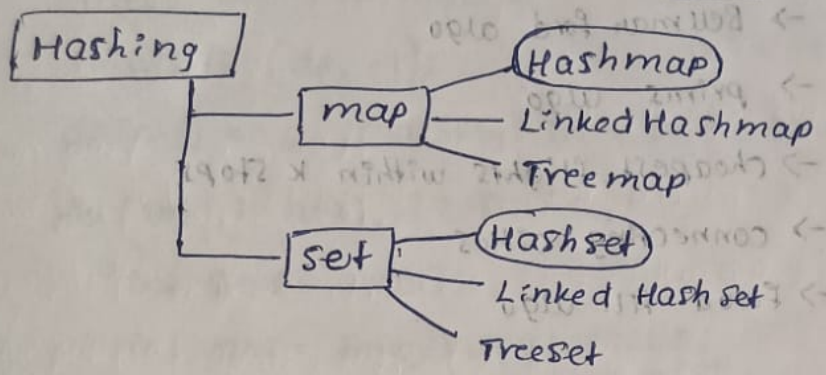
will the value change? Yes.



both are pointing to a same object.

* Hashing :

Hashing is a technique used to store & retrieve data very fast by converting a key into an index using a hash function. So data can be accessed in O(1) avg time.



→ Hash map :

Hash map is a datastructure that stores data in key-value pairs. uses hashing to allow fast insertion, deletion & searching.

(key , value)
↓
unique.

Tea	~~10~~ 20
Pizza	20
Samosa	50
coffee	20
↑ key	↑ value

insert, delete, search → O(1)

~~***~~ in hashmap key property is always unique. cannot duplicate. But values can duplicate like 20 & 20.
another Tea, pizza, samosa cannot be added, if you want to change the Tea price from 10 → 20.

To create a hashmap & it's operations:

```
import java.util.HashMap;

HashMap<String, Integer> hm = new HashMap<>();
```

↓ Tea, coffee (all strings)

↓ prices (Int)

* put(key, value); ⟶ put("chapati", 20);

```
hm.put("Tea", 10);
hm.put("chapati", 20);
hm.put("milk", 25);
```

System.out.println(hm);

[{Tea=10, chapati=20, milk=25}

your data will store in hashmap in a random order (unorder)

* get(key); ⟶ hm.get("Tea"); [10 // is a value

hm.get("coffee"); [null // not present

* containsKey("key"); ⟶ hm.containsKey("Tea"); [true

hm.containsKey("coffee"); [false

containsKey checks whether that key is exist in the hashmap & returns True or False.

* remove(key); ⟶ hm.remove("chapati"); [{Tea=10, milk=25}

it removes both the key & value present inside hashmap.

System.out.println(~~map~~ hm.remove("chapati")); ⟶ 20. it will print & deletes.

* size(); ⟶ hm.size();

* isEmpty(); ⟶ T or F.

* clear(); ⟶ hm.clear(); [True.

clear function clears (makes empty) complete hashmap and returns true.

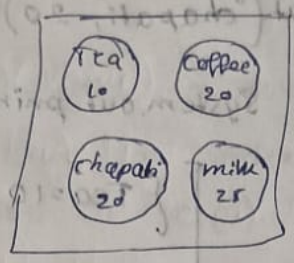
→ Iteration on Hashmap :

Set<String> keys = hm.keySet();

keySet func creates a set & stores all only keys inside that.

Type of the set can be change based on key type.

keySet → Set



hashmap.

```
hm.put("Tea", 10);
hm.put("coffee", 20);
hm.put("milk", 25);
hm.put("chapati", 20);
```

```
Set<String> keys = hm.keySet();
for(String k : keys){
    System.out.println(k + hm.get(k));
}
```

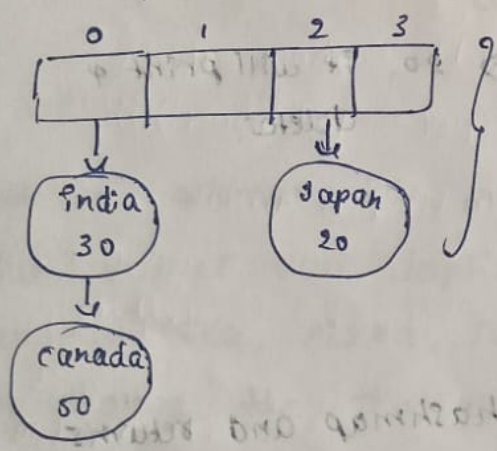
Tea 10 chapati 20
coffee 20
milk 25

Search for entrySet();

⇒ implementation of hashmap

put(), get(), containsKey(), remove(), size().

hashmap stores in a memory like array of LL. (linked list).



This is called as Bucket in hashing.

hashcode converts from one form to encrypted or a hashed formate, why this hashcode if you want to put any new data into hashmap, need to check key is exist or not exist, to find this you need to travel through all nodes which takes O(n). using hashcode we can perform in O(1).

here the work of hash function is take any key as a input & return the index ~~where~~ ~~if present~~.

key → [hash function] → index (bucketindex).

<u>implementation of hashmap code</u>:

```
import java.util.*;
public class HashMapCode{ // main class
    static class HashMap<K,V>{ // user defined class generic parameter.
                                  K & V. inside hashmap key can be
                                  any type so generic. (string, int, boo. etc)
// this class take 2 parameter.
private class Node{ // LL nodes, will store in each array index.
    K key;                  further array will be created.
    V value;
```

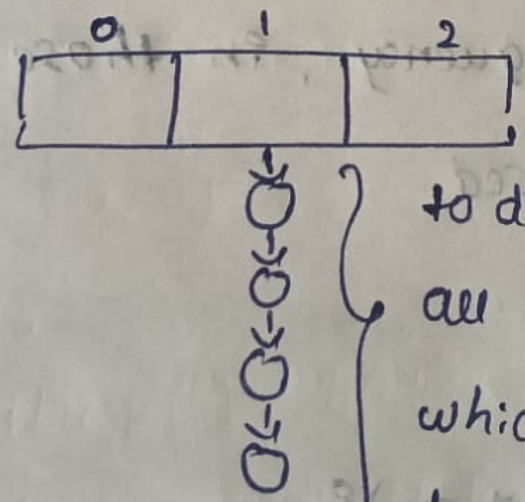
: Special Note

in hashing worst case time complexity becomes O(n). for all operations.

key → [hF] → 5 if hash function returning every time the same idx value, coincidentially (because hashcode is a random value).

all nodes will attach in the same idx.

key $\xrightarrow{HF}$ 1



to do all operations you travel through all nodes in worst case O(n). which is very rare but possible in hashmap.

→ Linked Hashmap :

* HashSet:

hashset is a collection that stores unique elements.

- no duplicate allowed
- no indexing
- insertion order not guranteed.
- HashSet internally use HashMap. but it stores only key.

```
HashSet<Integer> set = new HashSet<>();
```

Set operations:

```
set.add();      // adding elements.
set.remove();   // removing elements
set.contains();  // element exist or not in set.
set.size();     // find the total size.
```

all operations are O(1) constant.